

Rediff Chat: Enterprise Infrastructure & Integration Design

Comprehensive Blueprint: Multi-Substrate Storage, Distributed Message Buses, Dynamic Event-Flows & ejabberd Topology

1. EXECUTIVE SYSTEM OVERVIEW

Rediff Chat is a production-grade enterprise instant messaging platform built on top of the **ejabberd** XMPP engine. The platform implements a modern decoupled design, separating organizational corporate governance from real-time communication systems. User identity contexts, compliance archives, mobile tracking structures, and real-time transit telemetry are partitioned into highly specialized operational storage engines.

Rather than relying on direct cross-database dependencies, synchronization boundaries utilize a distributed **Asynchronous Event Bus infrastructure (Kafka / RabbitMQ)** along with high-performance runtime **External Authentication hooks**. This design isolates system runtimes, eliminates locking states, avoids single-point-of-failure vulnerabilities, and allows components to scale horizontally under enterprise-level chat workloads.



2. PRODUCTION DOMAIN OWNERSHIP MATRIX

Data ownership and state boundaries are tightly compartmentalized across four specialized database clusters and an independent out-of-band notification runtime:

DATA DOMAIN COMPONENT	OPERATIONAL OWNER	STORAGE SUBSTRATE	PRIMARY ARCHITECTURAL PURPOSE
User Identity Directory	Rediff Application	Rediff Core DB	Single system of record for global employee personnel structures.
Passwords / Credentials	Rediff Application	Rediff Core DB	Exclusive authentication target; eliminates multi-point credential sync vectors.
Personnel Profiles	Rediff Application	Rediff Core DB	Stores company business attributes (avatars, phone, designations).
Mobile Device Tokens	Rediff Application	Rediff Core DB	Maintains device maps needed for push notification targets.
XMPP Roster / Contacts	ejabberd Cluster	ejabberd Storage	Tracks buddy lists, mutual approval paths, and static groups.
Presence Real-time Telemetry	ejabberd Cluster	ejabberd Core Volatile	Manages ephemeral user status updates (online, away, dnd).
Active Sockets & Sessions	ejabberd Cluster	ejabberd Node Memory	Maintains active pipeline TCP/WebSocket client connections.
Message Routing Maps	ejabberd Cluster	ejabberd Node Memory	Internal cluster data paths for cross-node packet delivery.
Transient Offline Spools	ejabberd Cluster	ejabberd Storage	Short-term storage holding messages for offline recipients.
Persistent Chat Rooms (MUC)	ejabberd Cluster	ejabberd Storage	Group channel definitions, access privileges, and room names.
Security System Audit Logs	Audit Engine	Audit & Compliance	Immutable tracking ledger for administrative and access changes.
Historical Message Archives	Audit Engine	Audit & Compliance	Long-term store for compliance retention policies.
Delivery & Read Receipts	Messaging Pipeline	Delivery Store	Tracks real-time message status updates (Sent, Delivered, Read).
Push Notification Gateways	Notification Engine	Notification Service	Wakes dormant app containers via APNs and FCM endpoints.
Distributed Streaming Lines	Event Bus	Kafka / RabbitMQ	Asynchronous streaming broker connecting separate runtime segments.

3. SPECIALIZED STORAGE ARCHITECTURES & SCHEMA SPECS

Database 1: Rediff Core DB (Relational Identity Engine)

Manages long-lived identity details, cryptographic proofs, and mobile platform device maps.



Table: users [Primary Identity Block]

id (BIGINT, PK) | username (VARCHAR(100), UNIQUE) | email (VARCHAR(255), UNIQUE) | full_name (VARCHAR(255)) | status (VARCHAR(20)) | created_at (TIMESTAMP) | updated_at (TIMESTAMP)

Table: user_auth [Credential Store]

user_id (BIGINT, FK -> users.id, UNIQUE) | password_hash (VARCHAR(255)) | last_login_at (TIMESTAMP)

Table: user_profile [Directory Attributes]

user_id (BIGINT, FK -> users.id, UNIQUE) | phone (VARCHAR(50)) | avatar_url (VARCHAR(500)) | designation (VARCHAR(100))

Table: notification_devices [Out-of-band Target Routing Maps]

id (BIGINT, PK) | user_id (BIGINT, FK -> users.id) | platform (VARCHAR(20)) | device_token (TEXT) | created_at (TIMESTAMP) | updated_at (TIMESTAMP)

Database 2: ejabberd Core DB (Volatile Real-Time Substrate)

Optimized for rapid transaction execution, presence management, and runtime packet routing routes.

⚠ CRITICAL PRODUCTION DESIGN CHANGE: Because an External Authentication topology has been chosen, the native ejabberd credential storage repository table (**passwd**) is **NOT USED**. User credentials stay entirely within the Rediff Core DB.

Table: roster [Contact Storage – Mnesia Native Schema: [usj, us, jid, name, ...]]

owner_jid (VARCHAR(255), PK) | contact_jid (VARCHAR(255), PK) | subscription (VARCHAR(50)) | group_name (VARCHAR(100))

Table: session [Live Node Socket Telemetry]

`session_id` (VARCHAR(255), PK) | `jid` (VARCHAR(255)) | `resource` (VARCHAR(100)) | `login_time` (TIMESTAMP)

Table: route [Engine Internal Node Allocations]

`jid` (VARCHAR(255)) | `resource` (VARCHAR(100)) | `node` (VARCHAR(100)) | PK(`jid`, `resource`, `node`)

Table: offline_msg [Deferred Message Queue Buffer]

`message_id` (BIGINT, PK) | `sender_jid` (VARCHAR(255)) | `recipient_jid` (VARCHAR(255)) | `message_body` (TEXT) | `created_at` (TIMESTAMP)

Table: muc_room [Group Space Registry]

`room_jid` (VARCHAR(255), PK) | `room_name` (VARCHAR(255)) | `created_by` (VARCHAR(255)) | `created_at` (TIMESTAMP)

Table: muc_online_room [Active Group Space Status]

`room_jid` (VARCHAR(255), PK) | `active_users` (INT)

Database 3: Audit & Compliance Store (Immutable Corporate Ledger)

An append-only database isolated from real-time workflows to provide legally binding historical archives and tracking paths.

Table: `audit_log` [System Infrastructure Access Ledger]

`id` (BIGINT, PK) | `event_type` (VARCHAR(50)) | `actor_jid` (VARCHAR(255)) | `target_jid` (VARCHAR(255)) | `payload` (JSONB) | `created_at` (TIMESTAMP)

Table: `message_archive` [Historical Transcripts Retention]

`message_id` (VARCHAR(100), PK) | `sender_jid` (VARCHAR(255)) | `recipient_jid` (VARCHAR(255)) | `message_body` (TEXT) | `sent_at` (TIMESTAMP)

Database 4: Delivery Tracking Store (Real-Time State Engine)

Manages granular messaging states, serving as the source data engine for tracking receipts across all open chat tabs.

Table: `message_receipts` [State Progress Matrix]

`message_id` (VARCHAR(100)) | `recipient_jid` (VARCHAR(255)) | `status` (VARCHAR(20)) | `updated_at` (TIMESTAMP)

Valid Status Values: SENT, DELIVERED, READ

4. CROSS-SYSTEM IDENTITY TRANSLATION RULES

The architecture avoids explicit cross-database foreign keys. Synchronization handles depend entirely on a deterministic mapping convention processed at the event boundary lines:

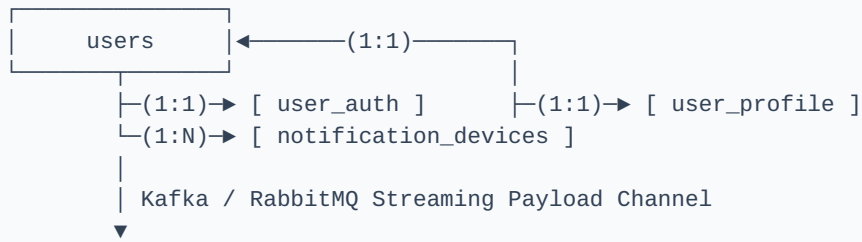
Rediff Core Directory (`users.username`) → Normalized String Rule → **Unified XMPP JID** (`username@chat.rediff.com`)

Example Integration: If an employee record contains internal directory identifier 101 and username string **alice**, the provisioning system maps all messaging telemetry down to the canonical XMPP target Jabber ID: alice@chat.rediff.com.

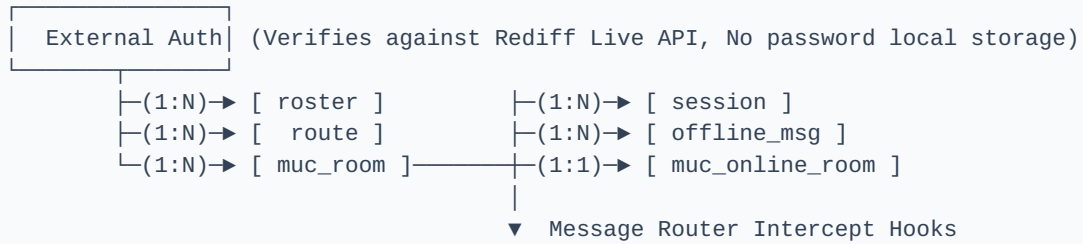
5. COMPLETE INTEGRATED RELATIONSHIP TOPOLOGY

The total mapping layout across all four data silos and decoupled workers shows a modern architecture without structural locking points:

[DATABASE 1: REDIFF SYSTEM OF RECORD]



[DATABASE 2: EJABBERD VOLATILE ENGINE]



[DATABASE 3 & 4: COMPLIANCE ARCHIVE & RECEIPT STORES]



6. EVENT BUS INFRASTRUCTURE & SCHEMA SPECS

Asynchronous operations are driven by publishing structured event payloads onto dedicated Kafka Topics or RabbitMQ Exchanges. This ensures the decoupling of system boundaries.

Structured Data Sample: UserCreated Payload

```
{
  "eventType": "UserCreated",
  "userId": 101,
  "username": "alice",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

Operational Event Distribution Matrix

SYSTEM STREAM EVENT	PIPELINE SOURCE ORIGIN	DOWNSTREAM CONSUMER CORE ACTION
UserCreated	INSERT INTO users database trigger	Triggers Provisioning Service to set up XMPP identifiers via the daemon.
UserDeleted	Corporate personnel removal pipeline	Removes active JIDs, tears down roster states, clears active node routes.
UserUpdated	Profile record modification update	Pushes structural profile data modifications down to XMPP vCards.
MessageSent	ejabberd engine router core hook	Triggers both Audit logging chains and the Push Notification service.
MessageDelivered	Client engine network package receipt	Updates message status state inside delivery tracking schemas to DELIVERED.
MessageRead	Inbound app viewport exposure signal	Updates state mapping inside delivery tracking schemas to READ.
RoomCreated / Deleted	MUC instance lifecycle adjustment event	Appends audit snapshots inside historical tracking tables.

7. PRODUCTION TECHNICAL EXECUTION FLOWS

A. Account Provisioning Chain

1. Rediff app commits write transaction to DB 1: INSERT INTO users (...).
2. App publishes a structured **UserCreated** event payload onto Kafka or RabbitMQ.
3. The independent Provisioning Service picks up the transaction packet from the stream.

4. Worker executes a secure shell command directly against the target ejabberd cluster engine daemon:

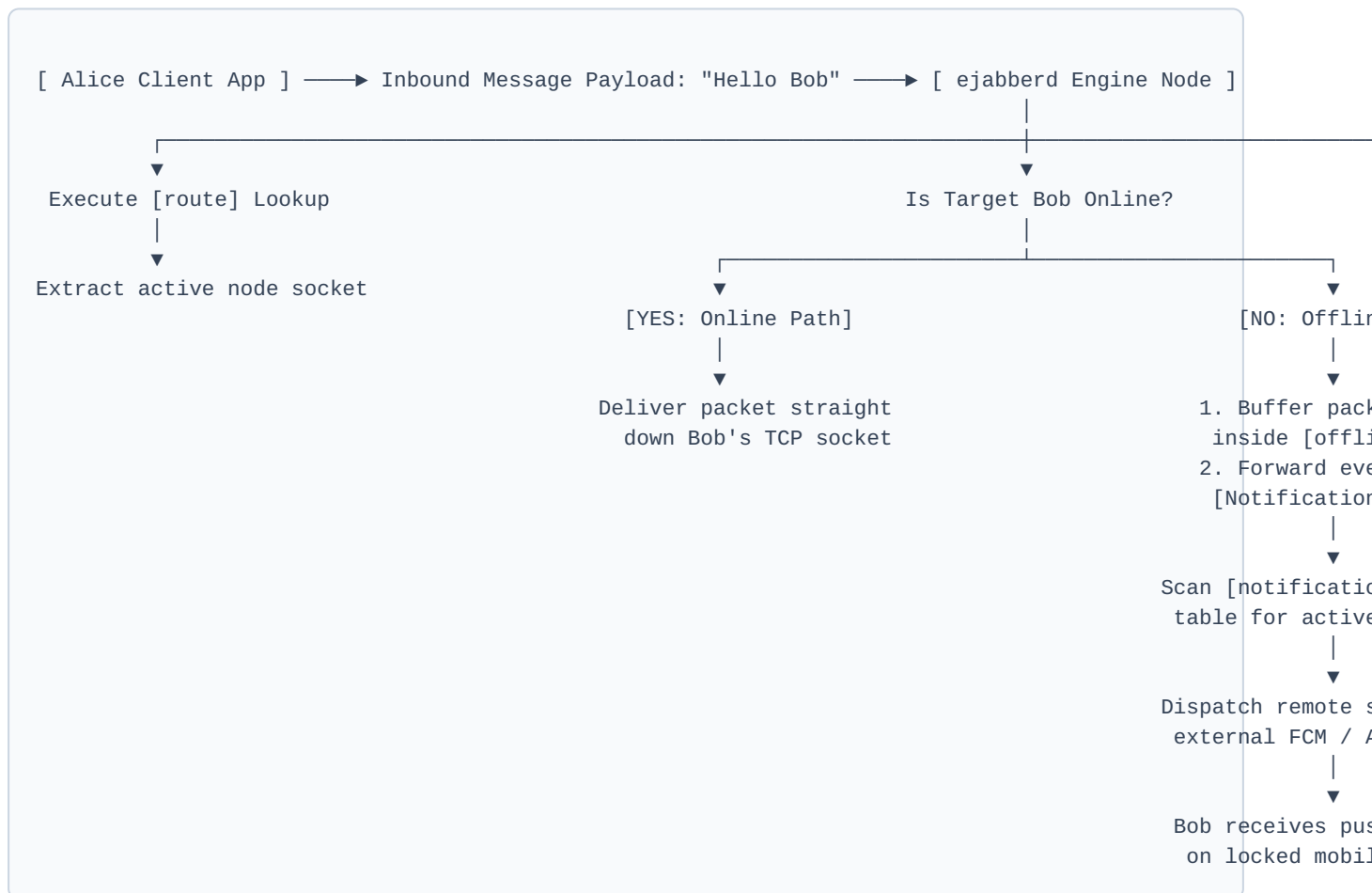
```
ejabberdctl register alice chat.rediff.com instance_secure_pass
```

B. Runtime Verification & Login Path

1. User passes identity checks (username + `password`) straight to the primary Rediff Auth Endpoint.
2. Server validates details against user_auth tables and returns a valid authorization token.
3. Chat client opens a secure web-socket link pointing directly to the ejabberd server cluster.
4. **Decoupled Verification Routing:** Instead of checking local storage tables, ejabberd fires its ****External Authentication Hook**** back to Rediff's Validation API in real-time. The token is confirmed, and connection access is authorized without passwords ever being stored locally inside ejabberd.

C. Core Message Distribution, Compliance, & Push Notification Path

This sequence outlines how the platform handles standard messaging, logs historical records, and triggers out-of-band mobile signaling when devices are sleeping:



D. Real-Time Delivery Receipt Mapping Flow

1. Bob launches the application container, triggering viewport exposure parameters on message lines.
2. The client app transmits a background receipt event packet labeled **MessageRead** to the server.
3. The ejabberd cluster engine intercept hook updates the status column inside the `message\_receipts` tracking matrix to **READ**.
4. The server signals Alice's active connection cluster node to display the terminal visual confirmation: ✓✓
Read.

8. CONSOLIDATED PRODUCTION DESIGN PRINCIPLES

1. Isolated Single-Source Credential Management

Rediff Core Database holds absolute domain authority over system logins, authentication tokens, directory metadata, and dynamic device tracking pools. Passwords are never duplicated inside the chat database tier.

2. Microsecond Runtime Comm-Engine Isolation

The ejabberd layer operates cleanly as an ephemeral messaging state container. Its storage layer is optimized purely for contact structures, volatile network presence mapping, group chat configurations, and short-term offline buffers.

3. Asynchronous Streaming Resiliency

Cross-system boundaries connect exclusively via a decoupled event bus. If compliance logging pipelines or remote notification gateways go down, transactions safely buffer within the message broker without causing lag in live chat operations.